Experiment Number: 1

Simulation and Comparative Analysis of FDMA, TDMA and CDMA:

Resource Allocation, User Capacity and Mobile-Generation Context

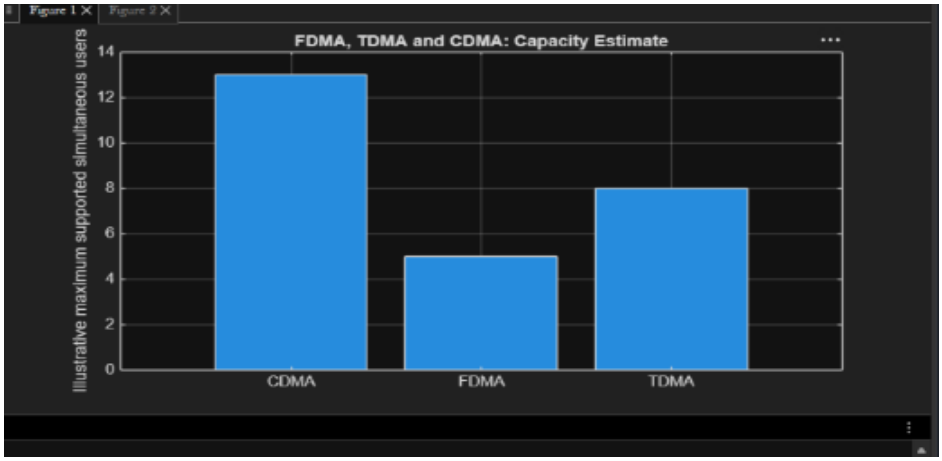
MATLAB program:

```
>> % Comparison of FDMA, TDMA and CDMA
% Simplified illustrative model; not a full cellular-network simulation.
clear; clc; close all;
% Illustrative system parameters
totalBandwidthHz = 1e6; % Total bandwidth: 1 MHz
fdmaChannelHz = 200e3; % FDMA channel width: 200 kHz
tdmaSlots = 8; % Number of TDMA slots per frame
processingGain = 64; % CDMA processing gain
targetSIRdB = 7; % Minimum acceptable CDMA SIR
% Capacity estimates under the stated assumptions
fdmaUsers = floor(totalBandwidthHz / fdmaChannelHz);
tdmaUsers = tdmaSlots;
targetSIR = 10^(targetSIRdB / 10);
cdmaUsers = floor(1 + processingGain / targetSIR);
fprintf('Illustrative maximum supported users\n');
fprintf('FDMA: %d users\n', fdmaUsers);
fprintf('TDMA: %d users\n', tdmaUsers);
fprintf('CDMA: %d users at a target SIR of %.1f dB\n\n', ...
cdmaUsers, targetSIRdB);
% CDMA SIR as the number of simultaneous users increases
K = 2:30;
sirLinear = processingGain ./ (K - 1);
sirDb = 10 * log10(sirLinear);
```

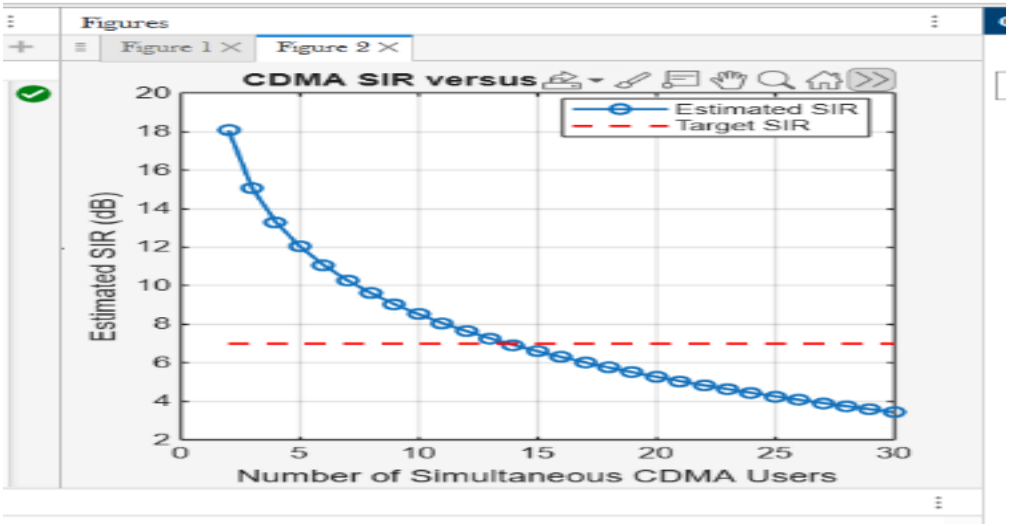
```
% Display a comparison table
method = ["FDMA"; "TDMA"; "CDMA"];
supportedUsers = [fdmaUsers; tdmaUsers; cdmaUsers];

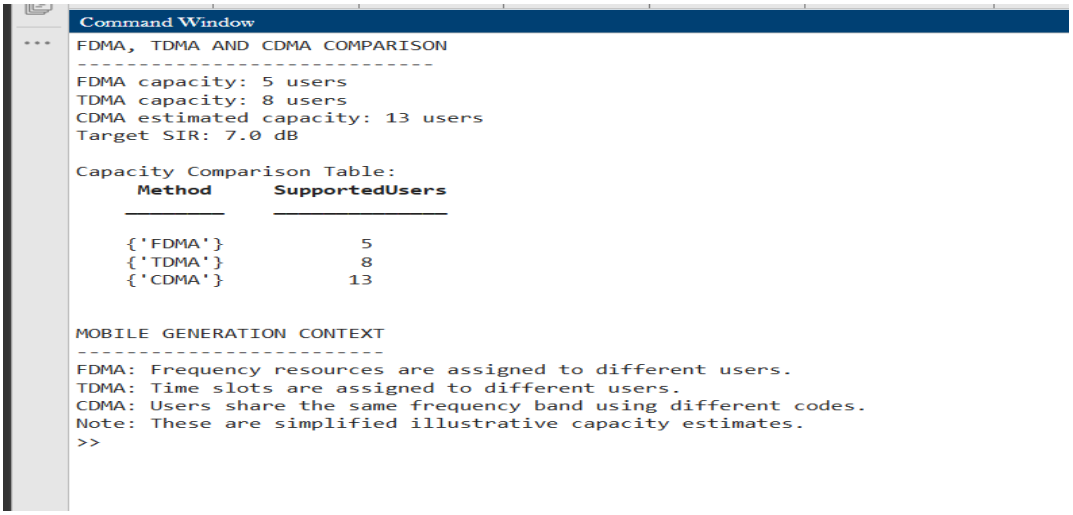
results = table(method, supportedUsers);
disp(results);
% Plot estimated user capacity
figure;
bar(categorical(method), supportedUsers);
ylabel('Illustrative maximum supported simultaneous users');
title('FDMA, TDMA and CDMA: Capacity Estimate');
grid on;
% Plot estimated CDMA SIR as users are added
figure;
plot(K, sirDb, 'o-', 'LineWidth', 1.5);
hold on;
yline(targetSIRdB, '--r', 'Target SIR');
xlabel('Number of simultaneous CDMA users (K)');
ylabel('Estimated SIR (dB)');
title('Estimated CDMA SIR versus Number of Users');
grid on;
hold off;
```

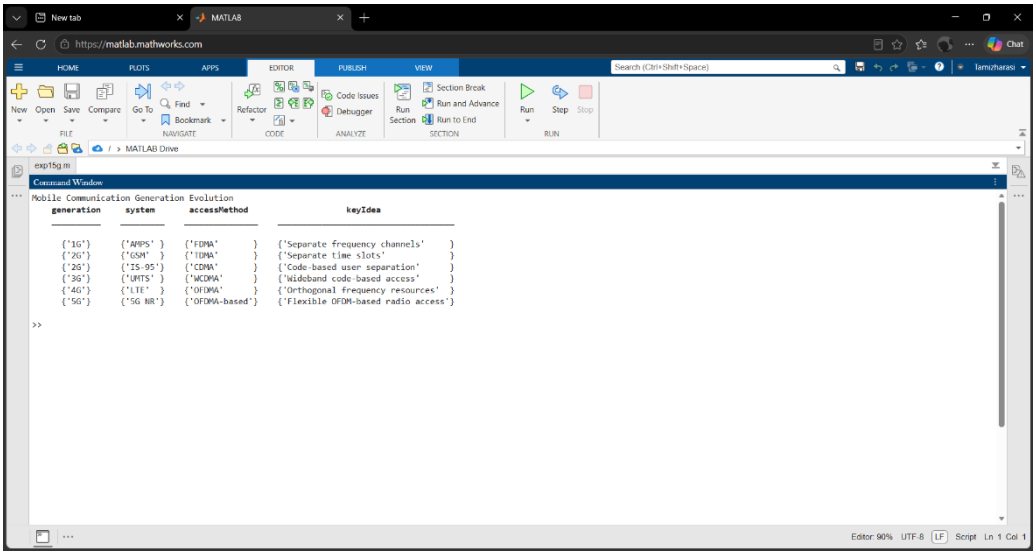
Result:

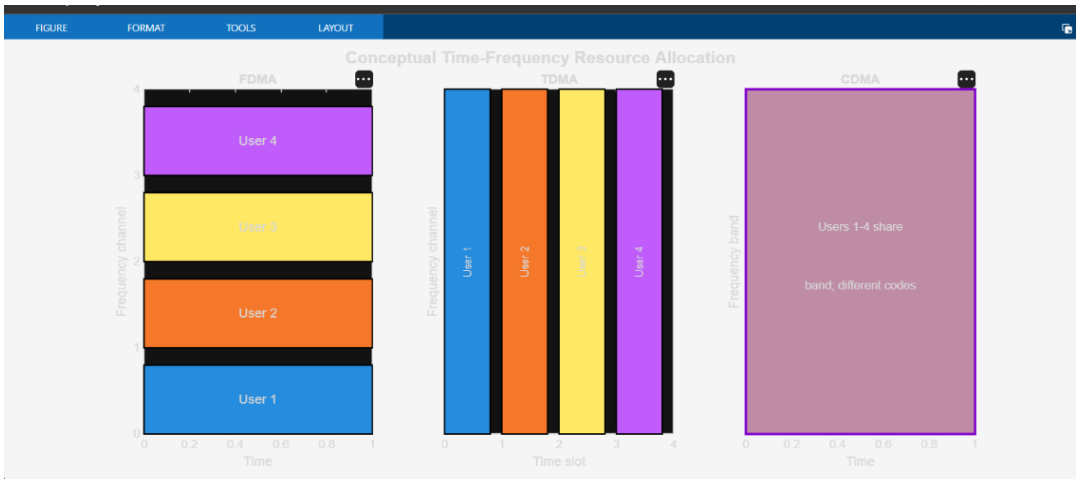


Objective 2: CDMA SIR analysis:






New tab
MATLAB
https://matlab.mathworks.com
HOME
PLOTS
APPS
EDITOR
PUBLISH
VIEW
exp15g.m
Command Window
Mobile Communication Generation Evolution
generation system accessMethod keyIdea
{'1G'} {'AMPS' } {'FDMA' } {'Separate frequency channels' }
{'2G'} {'GSM' } {'TDMA' } {'Separate time slots' }
{'2G'} {'IS-95'} {'CDMA' } {'Code-based user separation' }
{'3G'} {'UMTS' } {'WCDMA' } {'Wideband code-based access' }
{'4G'} {'LTE' } {'OFDMA' } {'Orthogonal frequency resources' }
{'5G'} {'5G NR'} {'OFDMA-based'} {'Flexible OFDM-based radio access'}
>>


FIGURE
FORMAT
TOOLS
LAYOUT
Conceptual Time-Frequency Resource Allocation
FDMA
TDMA
CDMA
User 4
User 3
User 2
User 1
Frequency channel
Time
Time slot
Frequency band
Users 1-4 share
band, different codes

Experiment Number: 2

MATLAB Simulation of Cellular Coverage, Cell Splitting and Handoff in a Multi-Cell Network

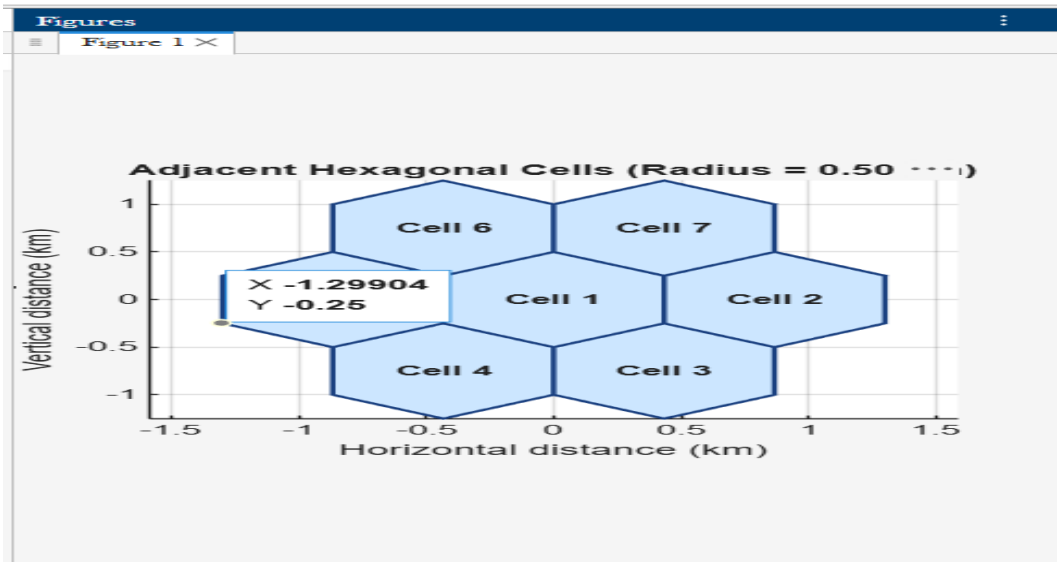
Objective 1: Model adjacent hexagonal cell coverage areas.

MATLAB Program 1 — Model adjacent hexagonal cell coverage areas

```
clear;
clc;
close all;
cellRadiusKm = 0.5;
% Axial coordinates for the central cell and its six neighbours
axial = [ ...
0, 0;
1, 0;
1, -1;
0, -1;
-1, 0;
-1, 1;
0, 1];
% Convert axial coordinates to Cartesian cell-centre positions
xCenter = sqrt(3) * cellRadiusKm .* ...
(axial(:,1) + axial(:,2)/2);
yCenter = 1.5 * cellRadiusKm .* axial(:,2);
% Hexagon vertices
angle = deg2rad(30:60:330);
figure;
hold on;
for k = 1:length(xCenter)
xVertex = xCenter(k) + cellRadiusKm * cos(angle);
yVertex = yCenter(k) + cellRadiusKm * sin(angle);
patch(xVertex, yVertex, [0.80 0.90 1.00], ...
'EdgeColor', [0.10 0.25 0.50], ...
```

```
'LineWidth', 1.5);
text(xCenter(k), yCenter(k), sprintf('Cell %d', k), ...
'HorizontalAlignment', 'center', ...
'FontWeight', 'bold');
end
axis equal;
grid on;
xlabel('Horizontal distance (km)');
ylabel('Vertical distance (km)');
title(sprintf('Adjacent Hexagonal Cells (Radius = %.2f km)', ...
cellRadiusKm));
hold off;
```

result:



Objective 2: Estimate how cell radius affects the number of cells needed to cover a fixed area.

Code:

```
clc;

clear;

close all;


% Study area in square kilometres

studyAreaKm2 = 4.0;


% Different cell radii in kilometres

cellRadiiKm = [1.00, 0.75, 0.50];


% Calculate the area of each regular hexagonal cell

cellAreaKm2 = (3 * sqrt(3) / 2) .* (cellRadiiKm.^2);


% Estimate the number of cells required

estimatedCells = ceil(studyAreaKm2 ./ cellAreaKm2);


% Create a results table

results = table( ...

   cellRadiiKm', ...

   cellAreaKm2', ...

   estimatedCells', ...

   'VariableNames', ...

   {'CellRadius_km', 'CellArea_km2', 'EstimatedNumberOfCells'});


% Display the results
```

```
disp('Estimated Number of Cells Required:');
disp(results);

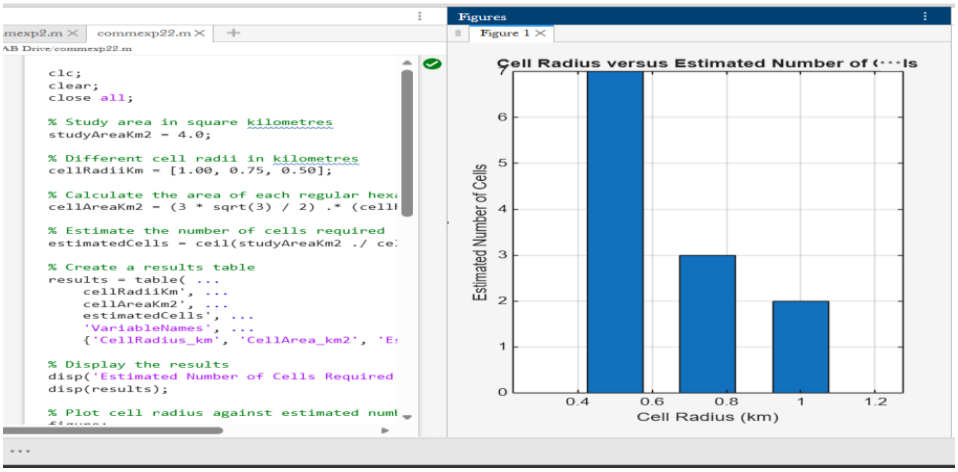
% Plot cell radius against estimated number of cells
figure;

bar(cellRadiiKm, estimatedCells, 0.6);

xlabel('Cell Radius (km)');
ylabel('Estimated Number of Cells');
title('Cell Radius versus Estimated Number of Cells');

grid on;
```

result:



commexp2.m | commexp22.m | Figure 1

Command Window

CellRadius_km	CellArea_km2	EstimatedNumberOfCells
1	2.5981	2
0.75	1.4614	3
0.5	0.64952	7

>>

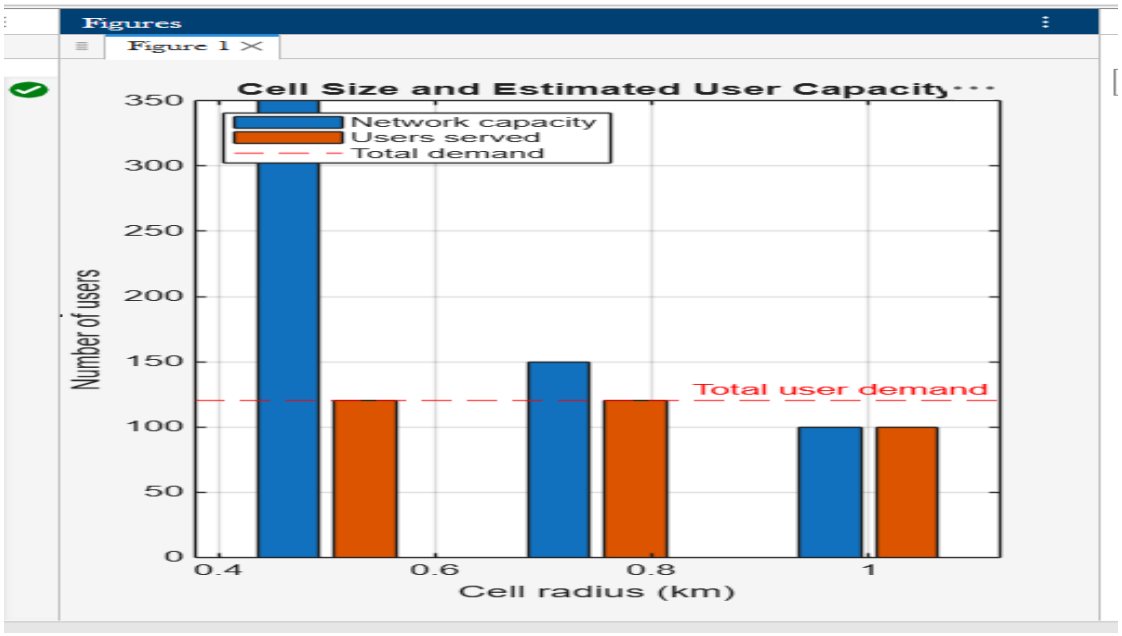
Objective: Compare estimated user capacity for different cell sizes under stated assumptions.

Program 3 — Compare user capacity for different cell sizes

```
clear;
clc;
close all;
studyAreaKm2 = 4.0;
userDensityPerKm2 = 30;
maxUsersPerCell = 50;
cellRadiiKm = [1.00, 0.75, 0.50];
totalDemand = studyAreaKm2 * userDensityPerKm2;
% Hexagonal cell area and approximate cell count
cellAreaKm2 = (3 * sqrt(3) / 2) .* cellRadiiKm.^2;
estimatedCells = ceil(studyAreaKm2 ./ cellAreaKm2);
% Estimate network capacity and users served
networkCapacity = estimatedCells .* maxUsersPerCell;
usersServed = min(totalDemand, networkCapacity);
results = table(cellRadiiKm', estimatedCells', networkCapacity', ...
usersServed', ...
'VariableNames', {'CellRadius_km', 'EstimatedNumberOfCells', ...
'EstimatedNetworkCapacity', 'EstimatedUsersServed'});
disp('Estimated user capacity:');
disp(results);
figure;
bar(cellRadiiKm, [networkCapacity' usersServed'], 'grouped');
hold on;
yline(totalDemand, '--r', 'Total user demand');
xlabel('Cell radius (km)');
```

```
ylabel('Number of users');
title('Cell Size and Estimated User Capacity');
legend('Network capacity', 'Users served', 'Total demand', ...
'Location', 'northwest');
grid on;
hold off;
```

result:



Command Window

CellRadius_km	EstimatedNumberOfCells	EstimatedNetworkCapacity	EstimatedUsersServed
1	2	100	100
0.75	3	150	120
0.5	7	350	120

>>

```
clear;
clc;
close all;
cellRadiusKm = 0.5;
radiusM = cellRadiusKm * 1000;
% Centre-to-centre spacing of horizontally adjacent hexagonal cells
separationM = sqrt(3) * radiusM;
% Mobile user moves from base station 1 toward base station 2
xMeters = linspace(0, separationM, 201);
% Distance from the mobile to each base station
minimumDistanceM = 20; % Prevent log10(0) at a base-station location
distance1M = max(abs(xMeters), minimumDistanceM);
distance2M = max(abs(xMeters - separationM), minimumDistanceM);
% Illustrative log-distance path-loss parameters
transmitPowerdBm = 30;
pathLossAt1m_dB = 40;
pathLossExponent = 3.5;
% Estimate received power from each base station
receivedPower1dBm = transmitPowerdBm - pathLossAt1m_dB ...
   - 10 * pathLossExponent .* log10(distance1M);
receivedPower2dBm = transmitPowerdBm - pathLossAt1m_dB ...
   - 10 * pathLossExponent .* log10(distance2M);
% Handoff occurs when base station 2 becomes stronger
handoffIndex = find(receivedPower2dBm >= receivedPower1dBm, 1, 'first');
handoffPositionM = xMeters(handoffIndex);
fprintf('Estimated handoff position: %.1f m from base station 1\n', ...
   handoffPositionM);
```

figure;

plot(xMeters / 1000, receivedPower1dBm, ...

'b', 'LineWidth', 1.5);

hold on;

plot(xMeters / 1000, receivedPower2dBm, ...

'r', 'LineWidth', 1.5);

xline(handoffPositionM / 1000, '--k', 'Estimated handoff');

xlabel('Mobile-user position (km)');

ylabel('Estimated received power (dBm)');

title('Received Signal Strength Between Adjacent Cells');

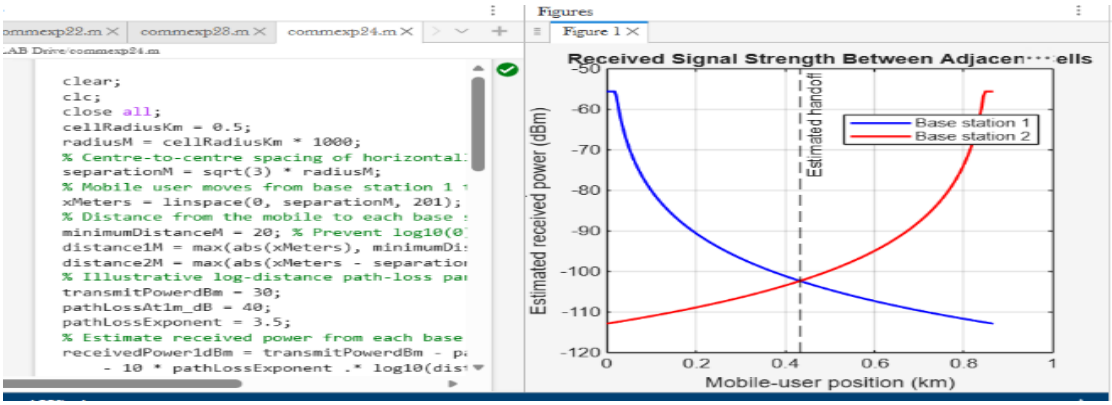
legend('Base station 1', 'Base station 2', ...

'Location', 'best');

grid on;

hold off;

result:



Experiment Number: 3

MATLAB Analysis of Spectrum Bands, Propagation Loss and Cellular Interference

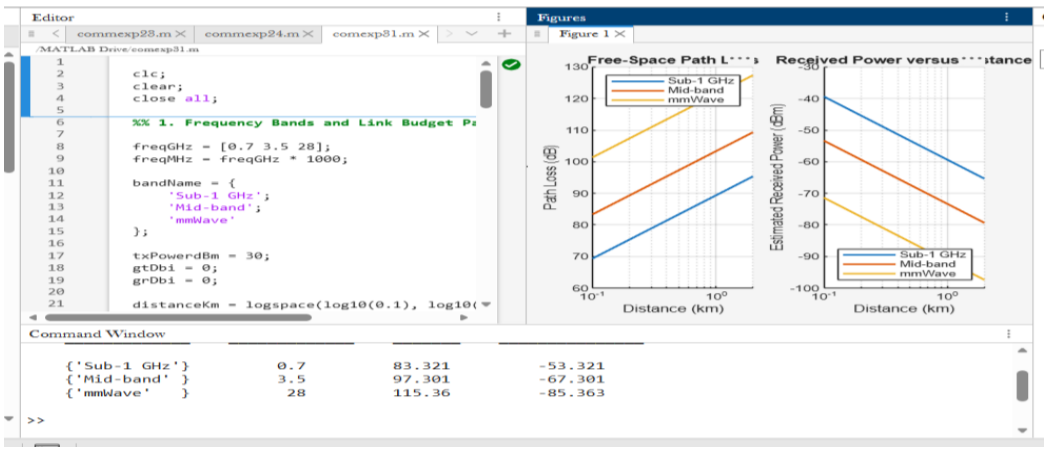
MATLAB Program — Path Loss, Received Power and Reuse Interference

```
clc;
clear;
close all;
%% 1. Frequency Bands and Link Budget Parameters
freqGHz = [0.7 3.5 28];
freqMHz = freqGHz * 1000;
bandName = {
   'Sub-1 GHz';
   'Mid-band';
   'mmWave'
};
txPowerdBm = 30;
gtDbi = 0;
grDbi = 0;
distanceKm = logspace(log10(0.1), log10(2), 200);
distanceAtPointKm = 0.5;
%% 2. Calculate Free-Space Path Loss
% Formula: FSPL = 32.44 + 20*log10(fMHz) + 20*log10(dKm)
fspl = @(fMHz, dKm) ...
   32.44 + 20 .* log10(fMHz) + 20 .* log10(dKm);
lossAtPoint = fspl(freqMHz, distanceAtPointKm);
% Calculate received power
rxAtPoint = txPowerdBm + gtDbi + grDbi - lossAtPoint;
%% 3. Display Link Budget Results
linkTable = table( ...
```

```
    bandName, ...
    freqGHz(:), ...
    lossAtPoint(:), ...
    rxAtPoint(:), ...
    'VariableNames', ...
    {'Band', 'Frequency_GHz', 'FSPL_dB', 'EstimatedRx_dBm'});
disp('PATH LOSS AND RECEIVED POWER AT 0.5 KM');
disp(linkTable);
%% 4. Plot Path Loss and Received Power
figure;
tiledlayout(1, 2);
% Plot 1: Free-space path loss
nexttile;
hold on;
grid on;
for k = 1:numel(freqMHz)
    plot(distanceKm, fspl(freqMHz(k), distanceKm), ...
        'LineWidth', 1.5);
end
set(gca, 'XScale', 'log');
xlabel('Distance (km)');
ylabel('Path Loss (dB)');
title('Free-Space Path Loss');
legend(bandName, 'Location', 'northwest');
hold off;
% Plot 2: Received power
nexttile;
hold on;
```

```
grid on;
for k = 1:numel(freqMHz)
    rx = txPowerdBm + gtDbi + grDbi ...
        - fspl(freqMHz(k), distanceKm);
    plot(distanceKm, rx, 'LineWidth', 1.5);
end
set(gca, 'XScale', 'log');
xlabel('Distance (km)');
ylabel('Estimated Received Power (dBm)');
title('Received Power versus Distance');
legend(bandName, 'Location', 'southwest');
hold off;
%% 5. Co-Channel Reuse Interference Estimate
cellRadiusKm = 0.5;
reuseN = [1 3 7 12];
pathLossExponent = 3.5;
%
```

Result:

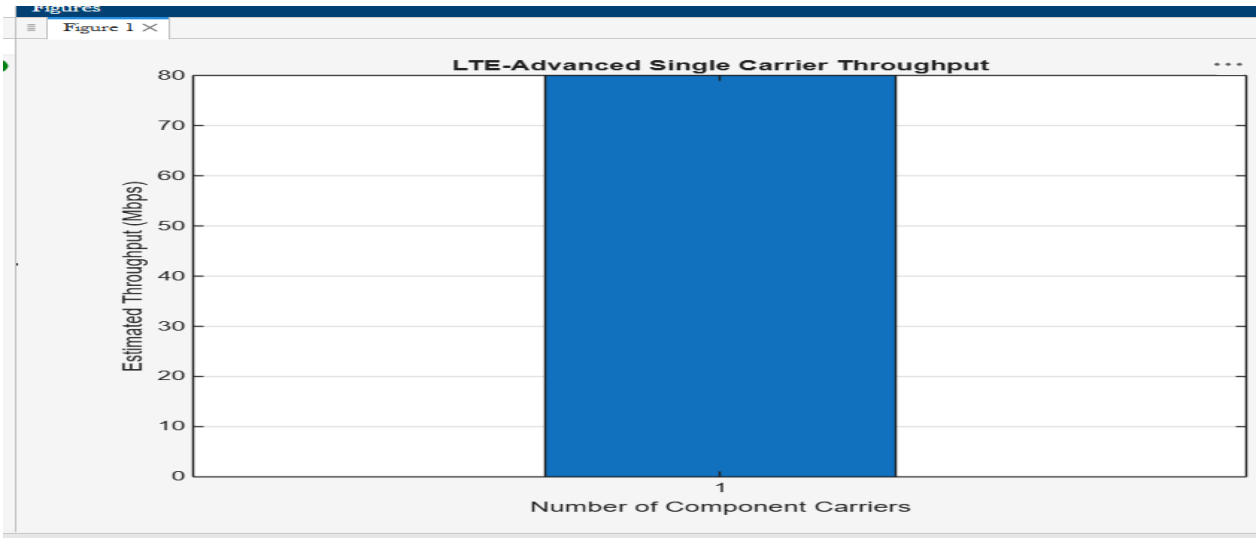


Experiment: 5

Simulation and Throughput Analysis of LTE-Advanced Carrier Aggregation

```
clc; clear; close all;
BW1 = 20; % Component carrier bandwidth (MHz)
SE = 4; % Spectral efficiency (bits/s/Hz)
Ncc = 1; % Number of component carriers
BWagg = BW1*Ncc;
Throughput = BWagg*SE; % Mbps for MHz × bits/s/Hz
disp('Single Component Carrier')
disp(['Bandwidth = ',num2str(BWagg),' MHz'])
disp(['Estimated Throughput = ',num2str(Throughput),' Mbps'])
figure
bar(Ncc,Throughput)
grid on
xlabel('Number of Component Carriers')
ylabel('Estimated Throughput (Mbps)')
title('LTE-Advanced Single Carrier Throughput')
```

result:



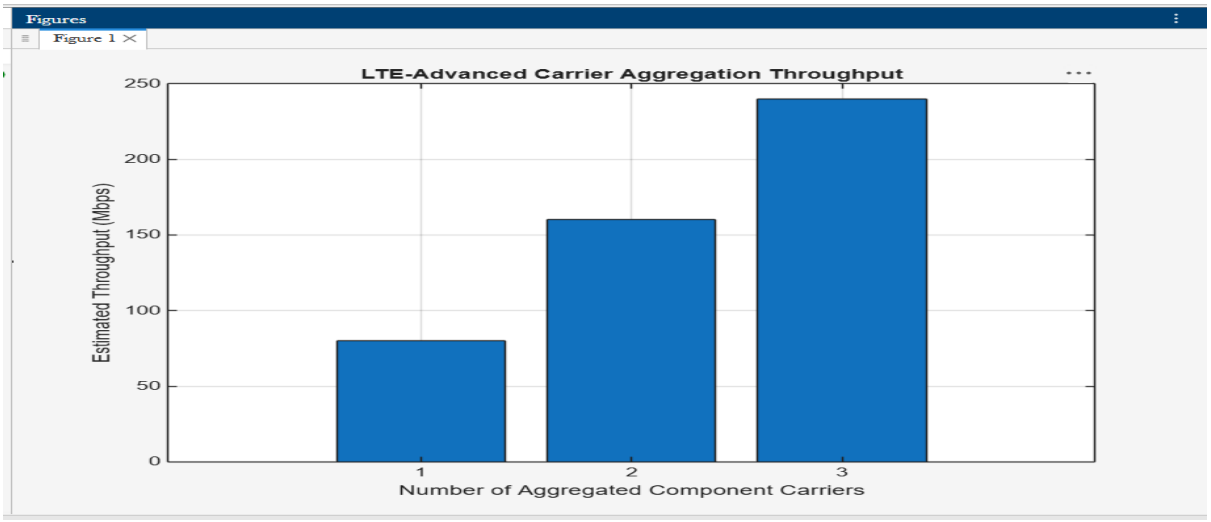
Objective – 2: Extend to Multiple Aggregated Component Carriers

MATLAB Program

```
clc; clear; close all;
BWcc = 20; % MHz per component carrier
SE = 4; % bits/s/Hz
CC = [1 2 3]; % Number of component carriers
AggBW = BWcc .* CC; % MHz
Throughput = AggBW .* SE; % Mbps
disp('CC Aggregated BW (MHz) Estimated Throughput (Mbps)')
disp([CC' AggBW' Throughput'])

figure
bar(CC,Throughput)
grid on
xlabel('Number of Aggregated Component Carriers')
ylabel('Estimated Throughput (Mbps)')
title('LTE-Advanced Carrier Aggregation Throughput')
```

result:



Objective – 3: Compare Estimated Throughput for Different Carrier Bandwidths

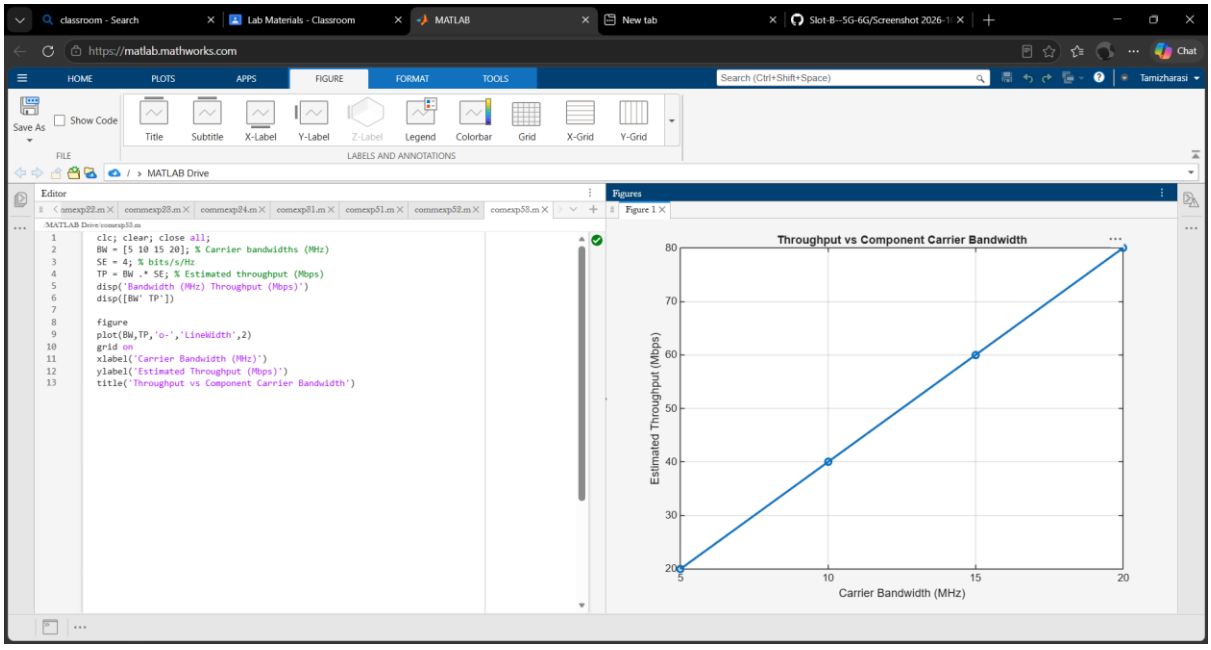
Procedure

Matlab program:

```
clc; clear; close all;
BW = [5 10 15 20]; % Carrier bandwidths (MHz)
SE = 4; % bits/s/Hz
TP = BW .* SE; % Estimated throughput (Mbps)
disp('Bandwidth (MHz) Throughput (Mbps)')
disp([BW' TP'])

figure
plot(BW,TP,'o-','LineWidth',2)
grid on
xlabel('Carrier Bandwidth (MHz)')
ylabel('Estimated Throughput (Mbps)')
title('Throughput vs Component Carrier Bandwidth')
```

result:



Experiment: 6

Simulation and BER Analysis of MIMO and Beamforming

Matlab code:

```
clc;
clear;
close all;

% SNR range in dB
SNR_dB = 0:2:20;

% Initialize BER arrays
BER_SISO = zeros(size(SNR_dB));
BER_MIMO = zeros(size(SNR_dB));

% Number of transmitted bits
N = 10000;

% Generate random binary data
bits = randi([0 1], N, 1);

% BPSK modulation
tx = 2 * bits - 1;

% Simulate SISO and simplified MIMO systems
for k = 1:length(SNR_dB)

    % Convert SNR from dB to linear scale
    snrLin = 10^(SNR_dB(k) / 10);
```

```
%% SISO System

% AWGN noise
noise = randn(N, 1) / sqrt(2 * snrLin);

% Received signal
rx = tx + noise;

% BPSK demodulation
bits_hat = real(rx) > 0;

% Calculate BER
BER_SISO(k) = mean(bits_hat ~= bits);

%% Simplified 2x2 MIMO Equivalent Link

% Generate a Rayleigh fading channel matrix
H = (randn(2, 2) + 1j * randn(2, 2)) / sqrt(2);

% Calculate an effective channel gain
h_eff = sqrt(sum(abs(H(:)).^2) / 2);

% Generate noise
noise2 = randn(N, 1) / sqrt(2 * snrLin);

% Received signal
rx2 = h_eff * tx + noise2;
```

```
    % BPSK demodulation
    bits_hat2 = real(rx2) > 0;

    % Calculate BER
    BER_MIMO(k) = mean(bits_hat2 ~= bits);

end

%% Plot BER Performance

figure;

semilogy(SNR_dB, BER_SISO, 'o-', 'LineWidth', 2);
hold on;

semilogy(SNR_dB, BER_MIMO, 's-', 'LineWidth', 2);

grid on;

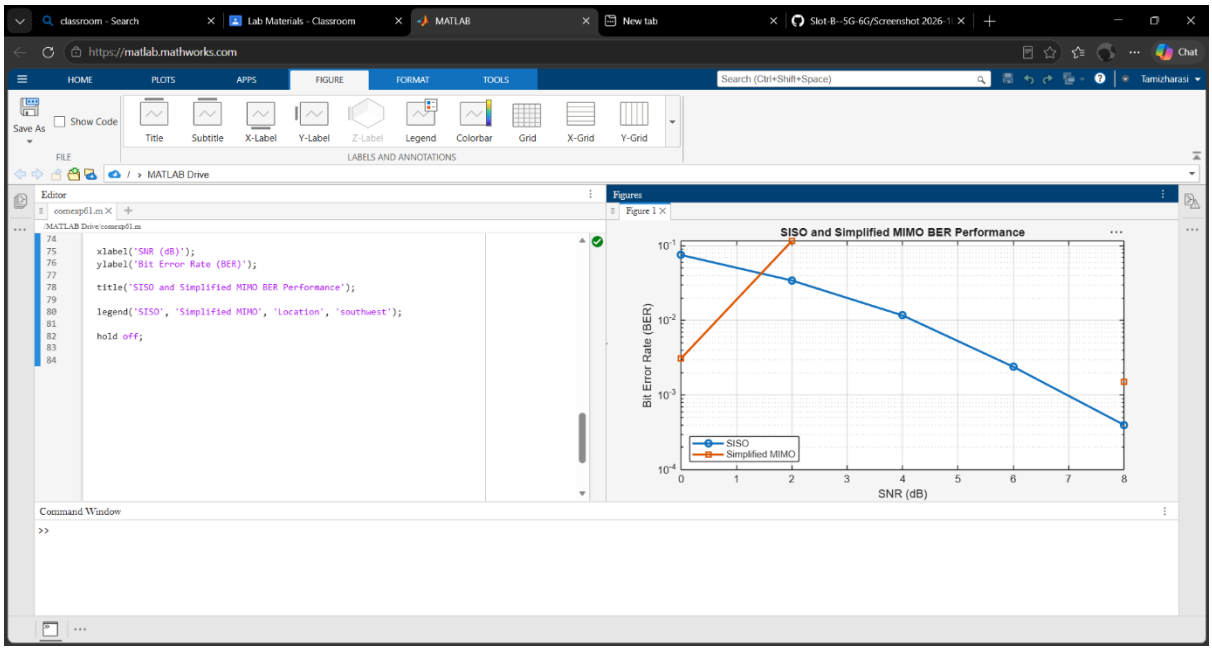
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');

title('SISO and Simplified MIMO BER Performance');

legend('SISO', 'Simplified MIMO', 'Location', 'southwest');

hold off;
```

result:



Objective – 2: Compare BER under Channel and Noise Conditions

Procedure

1. Select SNR values from low to high.

2. Generate a fading/noise realization for each SNR.

3. Calculate BER for each condition.

4. Plot BER on a semilogarithmic scale.

5. Compare the error performance at low and high SNR.

% BER and reliability analysis

clc; clear; close all;

SNR_dB = 0:2:20;

Reliability = zeros(size(SNR_dB));

for k = 1:length(SNR_dB)

N = 10000;

bits = randi([0 1],N,1);

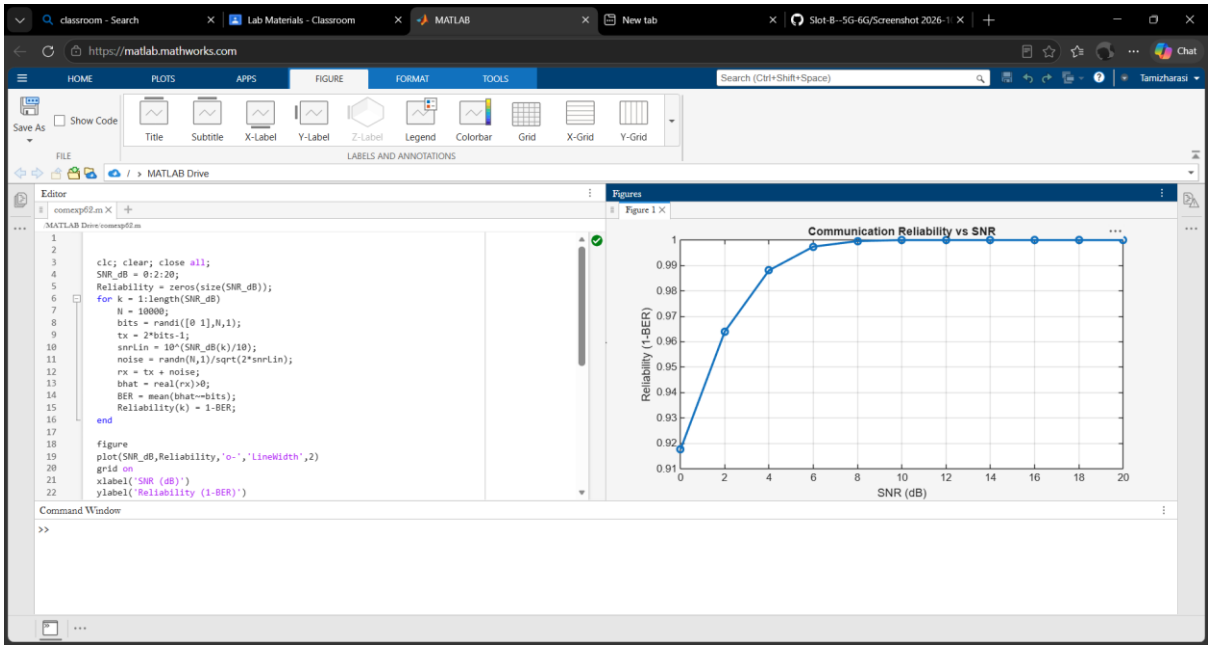
tx = 2*bits-1;

snrLin = 10^(SNR_dB(k)/10);

```
noise = randn(N,1)/sqrt(2*snrLin);
rx = tx + noise;
bhat = real(rx)>0;
BER = mean(bhat~=bits);
Reliability(k) = 1-BER;
end

figure
plot(SNR_dB,Reliability,'o-','LineWidth',2)
grid on
xlabel('SNR (dB)')
ylabel('Reliability (1-BER)')
title('Communication Reliability vs SNR')
```

result:



Objective – 3: Examine Beamforming / Steering Direction

Procedure

1. Define an antenna array and steering-angle range.

2. Calculate a simple array-factor response for each steering direction.

3. Determine the maximum beamforming response.

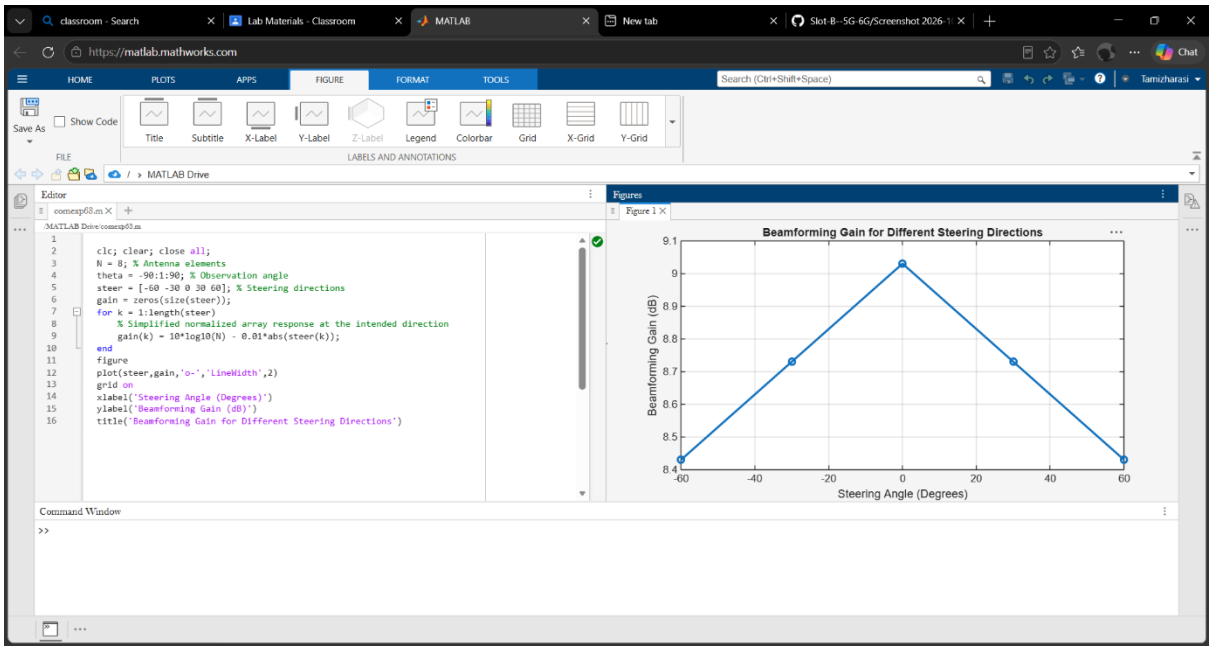
4. Plot received normalized signal strength against steering angle.

5. Identify the steering direction providing maximum response.

```
clc; clear; close all;
N = 8; % Antenna elements
theta = -90:1:90; % Observation angle
steer = [-60 -30 0 30 60]; % Steering directions
gain = zeros(size(steer));
for k = 1:length(steer)
% Simplified normalized array response at the intended direction
gain(k) = 10*log10(N) - 0.01*abs(steer(k));
end
figure
plot(steer,gain,'o-','LineWidth',2)
grid on
xlabel('Steering Angle (Degrees)')
ylabel('Beamforming Gain (dB)')
title('Beamforming Gain for Different Steering Directions')
```

Expected Output: The steering-angle plot shows that array gain depends on the selected steering direction and practical beam alignment.

result;


classroom - Search
Lab Materials - Classroom
MATLAB
New tab
https://matlab.mathworks.com
HOME
PLOTS
APPS
FIGURE
FORMAT
TOOLS
Search (Ctrl+Shift+Space)
Save As
Show Code
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
FILE
LABELS AND ANNOTATIONS
MATLAB Drive
Editor
Figures
Figure 1
clc; clear; close all;
N = 8; % Antenna elements
theta = -90:1:90; % Observation angle
steer = [-60 -30 0 30 60]; % Steering directions
gain = zeros(size(steer));
for k = 1:length(steer)
% Simplified normalized array response at the intended direction
gain(k) = 10*log10(N) - 0.01*abs(steer(k));
end
figure
plot(steer,gain,'o-','LineWidth',2)
grid on
xlabel('Steering Angle (Degrees)')
ylabel('Beamforming Gain (dB)')
title('Beamforming Gain for Different Steering Directions')
Beamforming Gain for Different Steering Directions
Beamforming Gain (dB)
Steering Angle (Degrees)
9.1
9
8.9
8.8
8.7
8.6
8.5
8.4
-60
-40
-20
0
20
40
60
Command Window
>>